

Lab 04 – IT23398184

Task 1

Code –

```
#include <GLUT/glut.h>
#include <stdlib.h>

// Activity 1 : createCube()
void createCube(void) {

    glClear(GL_COLOR_BUFFER_BIT);

    // Back (Green)
    glColor3f(0.0, 1.0, 0.0);
    glBegin(GL_POLYGON);
        glVertex3f(-0.2f, 0.0f, -0.4f); // bottom left
        glVertex3f(-0.2f, 0.4f, -0.4f); // top left
        glVertex3f( 0.2f, 0.4f, -0.4f); // top right
        glVertex3f( 0.2f, 0.0f, -0.4f); // bottom right
    glEnd();

    // Left (Blue)
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_POLYGON);
        glVertex3f(-0.4f, -0.2f, 0.0f);
        glVertex3f(-0.4f, 0.2f, 0.0f);
        glVertex3f(-0.2f, 0.4f, -0.4f);
        glVertex3f(-0.2f, 0.0f, -0.4f);
    glEnd();

    // Right (Blue)
    glBegin(GL_POLYGON);
        glVertex3f( 0.0f, -0.2f, 0.0f);
        glVertex3f( 0.0f, 0.2f, 0.0f);
        glVertex3f( 0.2f, 0.4f, -0.4f);
        glVertex3f( 0.2f, 0.0f, -0.4f);
    glEnd();

    // Top (White)
    glColor3f(1.0, 1.0, 1.0);
    glBegin(GL_POLYGON);
        glVertex3f(-0.4f, 0.2f, 0.0f);
        glVertex3f( 0.0f, 0.2f, 0.0f);
        glVertex3f( 0.2f, 0.4f, -0.4f);
```

```

        glVertex3f(-0.2f, 0.4f, -0.4f);
    glEnd();

    // Bottom (Yellow)
    glColor3f(1.0, 1.0, 0.0);
    glBegin(GL_POLYGON);
        glVertex3f(-0.4f, -0.2f, 0.0f);
        glVertex3f(0.0f, -0.2f, 0.0f);
        glVertex3f(0.2f, 0.0f, -0.4f);
        glVertex3f(-0.2f, 0.0f, -0.4f);
    glEnd();

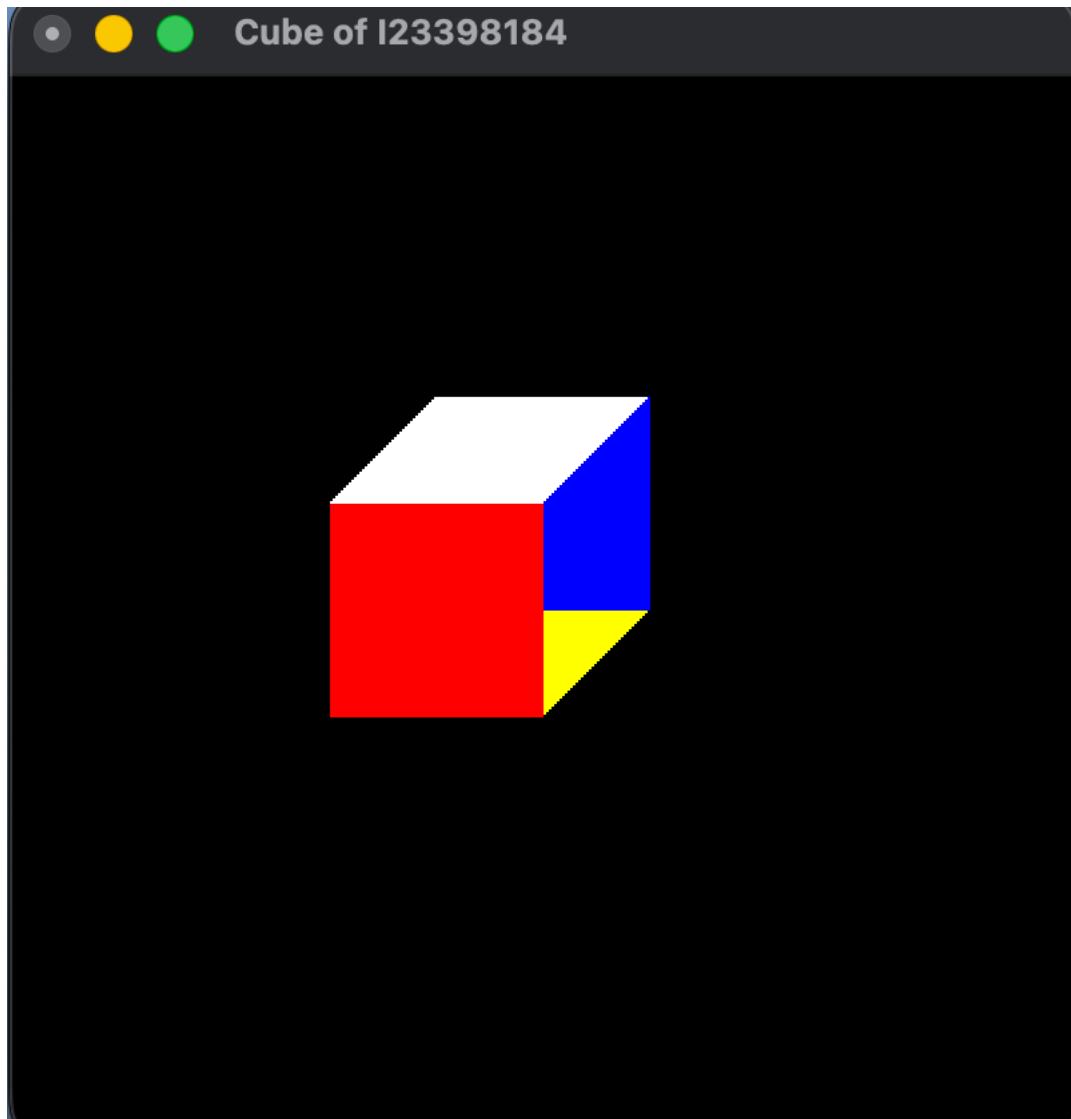
    // Front (Red)
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POLYGON);
        glVertex3f(-0.4f, -0.2f, 0.0f); // bottom left
        glVertex3f(-0.4f, 0.2f, 0.0f); // top left
        glVertex3f(0.0f, 0.2f, 0.0f); // top right
        glVertex3f(0.0f, -0.2f, 0.0f); // bottom right
    glEnd();

    // e) flush immediately
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv); // Initialize GLUT and process command-line arguments
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); // Set display mode with RGB color and
    single buffer
    glutInitWindowPosition(100, 100); // Set window position (upper-left corner at 200,
    200)
    glutInitWindowSize(400, 400); // Set window size (500x500 pixels)
    glutCreateWindow("Cube of I23398184"); // Create window with title
    glClearColor(0.0, 0.0, 0.0, 1.0); // Set the clear color (background color)
    to black
    glutDisplayFunc(createCube); // Register display callback function
    glutMainLoop(); // Enter the GLUT main loop
    return 0;
}

```

Output-



Task 2

Code-

```
#include <GLUT/glut.h>
#include <stdlib.h>

static float rotX = 0.0f;
static float rotY = 0.0f;

static void drawPyramid(void) {
    // Base (two triangles) - gray
```

```

glColor3f(0.5f, 0.5f, 0.5f);

glBegin(GL_TRIANGLES);
    glVertex3f(-0.5f, 0.0f, -0.5f);
    glVertex3f( 0.5f, 0.0f, -0.5f);
    glVertex3f(-0.5f, 0.0f,  0.5f);
glEnd();

glBegin(GL_TRIANGLES);
    glVertex3f( 0.5f, 0.0f, -0.5f);
    glVertex3f( 0.5f, 0.0f,  0.5f);
    glVertex3f(-0.5f, 0.0f,  0.5f);
glEnd();

// Sides to apex (0,1,0)
// Front (Red)
glColor3f(1.0f, 0.0f, 0.0f);
glBegin(GL_TRIANGLES);
    glVertex3f(-0.5f, 0.0f, -0.5f);
    glVertex3f( 0.5f, 0.0f, -0.5f);
    glVertex3f( 0.0f, 1.0f,  0.0f);
glEnd();

// Right (Green)
glColor3f(0.0f, 1.0f, 0.0f);
glBegin(GL_TRIANGLES);
    glVertex3f( 0.5f, 0.0f, -0.5f);
    glVertex3f( 0.5f, 0.0f,  0.5f);
    glVertex3f( 0.0f, 1.0f,  0.0f);
glEnd();

// Back (Blue)
glColor3f(0.0f, 0.0f, 1.0f);
glBegin(GL_TRIANGLES);
    glVertex3f( 0.5f, 0.0f,  0.5f);
    glVertex3f(-0.5f, 0.0f,  0.5f);
    glVertex3f( 0.0f, 1.0f,  0.0f);
glEnd();

// Left (Yellow)
glColor3f(1.0f, 1.0f, 0.0f);
glBegin(GL_TRIANGLES);
    glVertex3f(-0.5f, 0.0f,  0.5f);
    glVertex3f(-0.5f, 0.0f, -0.5f);
    glVertex3f( 0.0f, 1.0f,  0.0f);
glEnd();
}

// ---- display callback ----
static void display(void) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

```

```

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    // Move the scene a bit back so we can see it
    glTranslatef(0.0f, -0.2f, -2.2f);

    // Apply rotations from keyboard
    glRotatef(rotX, 1.0f, 0.0f, 0.0f);
    glRotatef(rotY, 0.0f, 1.0f, 0.0f);

    drawPyramid();

    glFlush();
}

// ---- optional: handle window resize, set projection ----
static void reshape(int w, int h) {
    if (h == 0) h = 1;
    glViewport(0, 0, (GLsizei)w, (GLsizei)h);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    // Simple perspective
    gluPerspective(60.0, (GLfloat)w / (GLfloat)h, 0.1, 100.0);

    glMatrixMode(GL_MODELVIEW);
}

// ---- keyboard: rotate or quit ----
static void keyboard(unsigned char key, int x, int y) {
    (void)x; (void)y; // unused
    switch (key) {
        case 'r': case 'R': rotY += 10.0f; break; // Y-axis
        case 'x': case 'X': rotX += 10.0f; break; // X-axis
        case 'b': case 'B': // change background to blue (demo)
            glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
            break;
        case 'q': case 'Q': case 27: // Esc
            exit(0);
            break;
        default: break;
    }
    glutPostRedisplay();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    // Depth buffer for correct face overlap
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowPosition(200, 200);

```

```
glutInitWindowSize(500, 500);  
glutCreateWindow("Pyramid - IT23398184"); // replace with your IT index  
  
// Background (black initially)  
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);  
glEnable(GL_DEPTH_TEST);  
  
glutDisplayFunc(display);  
glutReshapeFunc(reshape);  
glutKeyboardFunc(keyboard);  
  
glutMainLoop();  
return 0;  
}
```

Output-

